

CONTENTS

- Overview
- 1 Why the same question can have two correct answers
- 2 Real document sets accumulate contradictions over time
- 3 The baseline RAG pipeline: retrieve, then generate
- 4 Fix 1: Don't let stale documents into the index (avoid unforced errors)
- 5 Fix 2: Use a clarification loop to catch bad or underspecified questions
- 6 Fix 3: Calibrate the AI's confidence to what the data actually supports
- 7 Know your data before you build: 'hallucination' is often a design flaw
- Glossary
- Check yourself
- Where to go next

Why RAG Solutions Fail with Complex Documents & Vector Databases

Learn why retrieval-augmented generation breaks down on ambiguous, contradictory document sets, and how to design around it.

For engineers building RAG systems on real-world document collections that grow, get revised, and sometimes contradict themselves · 27 July 2026

[Watch the source video](#)[Read the condensed version](#)[Read the highlights](#)

BEFORE YOU START

- Basic understanding of how a retrieval-augmented generation (RAG) pipeline works end to end
- Familiarity with the idea of a vector database and semantic search

Overview

Retrieval-augmented generation (RAG) systems are usually demoed on clean, single-source-of-truth data, where every question has exactly one correct answer sitting in exactly one document. Real document collections rarely look like that. Policies get revised, laws get amended, and the same underlying fact gets described differently depending on who wrote it and when. A RAG system built for the clean case will confidently produce wrong, contradictory, or misleadingly certain answers when it meets the messy case.

This lesson walks through why that happens and three concrete design changes that address it: curating what actually lives in the vector database, adding a clarification step before retrieval, and calibrating the system's confidence to match what the source documents genuinely support. The throughline is that most of these failures are not the model 'hallucinating' — they are the system being designed as if the underlying data had one answer when it does not.

KEY TAKEAWAYS

- ✓ Natural-language questions often carry an unstated scope (like 'season year' versus 'the calendar year the championship game was played'), so two answers can both be correct for slightly different implied questions.
- ✓ Real document corpora accumulate contradictions over time because they are written and revised by many people across years, without a single author reconciling old and new claims.
- ✓ A RAG pipeline is only as trustworthy as its index: stale or superseded documents left in a vector database get retrieved and mixed into the model's context just like current ones.
- ✓ Tagging documents with effective dates and supersession links lets you filter out obsolete content before it ever reaches the model, preventing what the video calls an 'unforced error.'
- ✓ A clarification loop lets the system catch nonsensical or underspecified questions and ask the user for more detail instead of guessing at intent.
- ✓ The system's confidence must not exceed what the data supports: if a careful human reader of the source documents would say 'it depends,' the AI's answer should say that too, not commit to a single value.
- ✓ Opinion-based sources, such as legal opinions, should be presented as opinions in their original context, not restated as settled fact.
- ✓ Many reported 'hallucinations' are actually a mismatch between the corpus's true epistemic status (contested versus single-answer) and a design that assumed there was only one right answer.

01 Why the same question can have two correct answers

0:00

Language is ambiguous about scope, so two people (or two documents) can give different answers that are each correct for a slightly different implied question.

Consider asking who won the 2020 NCAA football national championship. One person says LSU; another says Alabama. Neither is lying. The LSU football season that started in August 2019 ended with a championship game played in January 2020, so people casually refer to that team as both the '2019 champion' (the season) and the '2020 champion' (the year the trophy was actually awarded). The question 'who won in 2020' is ambiguous about which of those two meanings it intends, and that ambiguity lives in the question itself, not in any error in either answer.

This matters for RAG because a vector database does not resolve ambiguity — it retrieves whatever text is semantically closest to the query. If your document set contains language using both conventions, a query about '2020 champion' can legitimately surface passages supporting either team. A system that returns a single flat answer is quietly picking one interpretation of the question without telling the user it did so, which is worse than being wrong: it looks confident while being arbitrary.

KEY POINTS

- Ambiguity often hides an implicit scope (e.g., season year vs. the calendar year of the deciding event) that the asker never stated.
- Two answers can both be factually correct if they are answering two different, unstated versions of the question.
- A RAG system that returns one flat answer strips away the context needed to reconcile competing but valid interpretations.
- The fix starts with recognizing the ambiguity exists, not with picking a 'more correct' answer.

The LSU championship-year ambiguity

LSU's championship-winning football season ran from August 2019 to the title game in January 2020. Common usage calls them both the '2019 champion' and the '2020 champion' depending on whether the speaker means the season or the year the game was played, and a document set written by many people over time will contain both phrasings.

02 Real document sets accumulate contradictions over time

1:23

Because documents are authored and revised by different people across years, a large repository almost always contains claims that were true once and are no longer, alongside claims that are still current.

A single well-maintained document rarely contradicts itself. A repository of hundreds or thousands of documents, written by different people at different times for different purposes, almost always does. A policy written in the 1990s can conflict with one written in the 2020s on the same topic. A law passed in 1912 can conflict with one passed in 2012. None of this is a data-quality bug to be 'fixed' in the traditional sense — it is an accurate reflection of how institutions actually produce and revise documents.

The consequence for RAG is that semantic search, working as designed, will retrieve chunks from both the old and the new document if they both discuss the same topic in similar language. The retrieval step is not broken; it is faithfully surfacing everything relevant. The problem only appears one step later, when that mixed, sometimes-contradictory context gets handed to an LLM that is asked to produce one confident answer.

KEY POINTS

- Multiple authors writing across years is normal, and it naturally produces documents that disagree with each other.
- Retrieval doing its job (surfacing everything relevant) is exactly what causes both old and new claims to appear together.
- 'Correct' in a large document set is frequently 'correct as of a specific date,' not correct in an absolute sense.
- The contradiction only becomes a visible failure once an LLM is forced to synthesize a single answer from conflicting context.

Federal vs. state regulation conflicts

An agency governed by both federal and state regulations will often have rules that contradict one another outright, simply because they were written by different bodies at different times for different jurisdictions. A retrieval system searching that corpus has no way to know which rule the user actually needs without more context.

03 The baseline RAG pipeline: retrieve, then generate

2:06

A minimal RAG system embeds the user's question, retrieves the closest-matching chunks from a vector database, and hands both the question and the retrieved text to an LLM to produce the answer.

The simplest RAG architecture has three moving parts. First, a user asks a question in natural language. Second, that question is converted into an embedding — a numeric vector that captures its meaning — and used to search a vector database, which returns the stored text chunks whose embeddings are closest to the query's embedding (a nearest-neighbor search over meaning rather than exact keywords). Third, those retrieved chunks and the original question are both passed into an LLM, which reads the retrieved context and generates an answer grounded in it.

The key limitation of this baseline design is that the LLM only ever sees what retrieval hands it. It has no independent way to know that one retrieved chunk is a superseded policy and another is its replacement, or that one is an opinion and another is a stated fact. Retrieval quality caps answer quality: if the retrieved set is messy, contradictory, or missing important context, the generation step inherits all of that, and no amount of prompting can fully compensate.

KEY POINTS

- Embeddings turn text into vectors so that semantically similar text ends up numerically close, enabling meaning-based rather than keyword-based search.
- The vector database's job is nearest-neighbor search: return the k chunks most similar to the query.
- The LLM's context window is filled entirely by what retrieval selected — it cannot see documents it wasn't given.
- A naive pipeline treats every retrieved chunk as equally current and equally trustworthy, which is rarely true for real corpora.

A minimal RAG loop

A basic implementation embeds the question, retrieves the top-k nearest chunks from a vector store, stuffs them into a prompt as context, and asks the LLM to answer using only that context.

```
def naive_rag_answer(question, vector_db, llm):
    query_embedding = embed(question)
    top_chunks = vector_db.search(query_embedding, k=5)
    context = "\n\n".join(chunk.text for chunk in top_chunks)
    prompt = (
        "Answer the question using only the context below.\n\n"
        f"Context:\n{context}\n\n"
        f"Question: {question}"
    )
    return llm.generate(prompt)
```

04 Fix 1: Don't let stale documents into the index (avoid unforced errors)

2:51

Leaving superseded documents in the vector database is a preventable mistake, not an inherent limitation of RAG — treat document curation as part of the pipeline, not an afterthought.

An 'unforced error' here means a mistake you could have avoided through better document management, as opposed to a genuine limitation of retrieval or generation. Suppose a policy was implemented in 2019 and replaced by a new policy in 2024. If both documents sit in the vector database with no indication that one supersedes the other, a query about that policy can retrieve chunks from both, and the LLM has no principled way to prefer the current one. The confusion isn't a model failure; it's a failure to curate what the model was allowed to see.

The fix is to make document lifecycle management part of the RAG system itself, not a one-time cleanup task. Every document should carry metadata about when it took effect, whether it has been superseded, and by what. That metadata can then be used to filter or downweight obsolete content either at ingestion time (never index the old version) or at query time (exclude superseded documents from search results). Garbage curation produces garbage answers no matter how good the underlying LLM is.

KEY POINTS

- An 'unforced error' is a preventable mistake in what you put into the vector database, distinct from a limitation of retrieval or generation.
- Superseded documents left in the index get retrieved alongside current ones with no signal that they're outdated.
- Attaching effective-date and supersession metadata to every document lets you filter obsolete content programmatically.
- Document curation is part of the RAG pipeline's design surface, not a separate data-hygiene chore.

Metadata-filtered retrieval

Each document stores an `effective_date`, a `status`, and an optional `superseded_by` pointer. At query time, the search call filters out any document whose status is 'superseded' so only the currently active version can be retrieved.

```
document_metadata = {
    "doc_id": "policy-2019-07",
    "effective_date": "2019-01-15",
    "status": "superseded",
    "superseded_by": "policy-2024-03"
}

# At query time, only search documents that are still active
results = vector_db.search(
    query_embedding,
    k=5,
    metadata_filter={"status": "active"}
)
```

Fix 2: Use a clarification loop to catch bad or underspecified questions

3:31

Before retrieving anything, check whether the question is answerable and specific enough — if not, ask the user to clarify instead of guessing.

A clarification loop is a check inserted before (or instead of) direct retrieval, whose job is to make sure the system is answering the question the user actually meant. It handles two distinct failure modes. The first is nonsensical input: a question like 'why are banana?' doesn't map to anything meaningful in the domain, and the system should recognize that and ask the user to rephrase rather than force an answer out of whatever text happens to be nearest in the vector space.

The second, subtler failure mode is underspecification. A question like 'who won the championship in 2010?' is grammatically fine but missing critical scope — championship of what? Sports, spelling, something else entirely? Without that scope, retrieval will return whatever is closest by coincidence, and the answer will be arbitrary rather than correct. A clarification loop catches this by checking, before searching, whether the question contains enough information to identify a unique, well-defined answer, and if not, prompts the user for the missing detail.

KEY POINTS

- Classify the question before retrieving, not after — catching problems early avoids generating a confident answer to a bad question.
- Nonsense detection prevents the system from forcing an answer out of loosely related retrieved text.
- A specificity check catches queries missing required scope, such as a domain or category the question assumed but never stated.
- A clarification loop turns an ambiguous query into a short dialogue instead of a silent guess.

A clarification classifier prompt

Before running retrieval, a lightweight LLM call classifies whether the incoming question is well-formed and specific enough to answer, returning a structured decision the application can act on.

```
CLARIFICATION_PROMPT = """
```

```
You are checking a question before it is answered by a document search system.
```

```
Return JSON with three fields:
```

- "needs_clarification": true or false
- "reason": why, if clarification is needed
- "follow_up_question": what to ask the user, if needed

```
A question needs clarification if it is nonsensical, or if it is missing  
scope needed to identify a single well-defined answer (e.g. a domain,  
category, jurisdiction, or time period).
```

```
Question: \"{question}\"
```

```
"""
```

```
def check_before_retrieval(question, llm):
```

```
    result = llm.generate(CLARIFICATION_PROMPT.format(question=question))
```

```
    return json.loads(result)
```

06 Fix 3: Calibrate the AI's confidence to what the data actually supports

4:55

The system's certainty should never exceed what a careful human reader of the same documents would conclude — if the data supports X, Y, or Z, the answer should say so.

This is presented as the most important and most commonly violated principle: the AI has to be less than or equal to your data. If a human read every relevant document in the corpus and would conclude 'the answer could be X, Y, or Z,' then the AI's answer needs to reflect that same multiplicity. A system that collapses a genuinely multi-answer question into a single confident value isn't being helpful — it's being wrong in a way that's harder to catch, because it sounds authoritative.

This principle applies with particular force to policies, laws, and regulations, where it's common for older and newer rules to coexist and contradict, and to corpora built from opinions rather than settled fact, such as legal opinions. In that case, the system should present the opinion as an opinion, attributed and contextualized, rather than restating it as a determined fact. Getting this right requires changing how answers are synthesized, not just how documents are retrieved: the generation prompt itself needs to instruct the model to surface disagreement rather than resolve it artificially.

KEY POINTS

- Forcing a single answer when the corpus genuinely supports several is a design defect, not a limitation of the underlying model.
- When multiple valid answers exist, present them together with their sourcing rather than silently picking one.
- Distinguish fact-type corpora from opinion-type corpora, and label outputs accordingly.
- This fix lives in the answer-synthesis prompt and logic, not only in what gets retrieved.

A synthesis prompt that preserves disagreement

Instead of instructing the LLM to produce one answer, the prompt explicitly tells it to enumerate every answer the retrieved context supports, cite which document backs each one, and flag opinion-based sources as opinions rather than facts.

```
SYNTHESIS_PROMPT = """
```

```
Using only the context below, answer the question.
```

```
If the context supports more than one answer (for example, because documents from different dates disagree), list every supported answer separately with the source and date for each. Do not pick one unless the context makes clear it is the only current, applicable answer.
```

```
If a source is an opinion or interpretation rather than a stated fact, label it as such and attribute it explicitly.
```

```
Context:
```

```
{context}
```

```
Question: {question}
```

```
"""
```

Know your data before you build: 'hallucination' is often a design flaw

6:19

Understand whether your corpus is single-answer, contested, or opinion-based before choosing how the system should answer — mismatches here get misdiagnosed as hallucination.

The recurring theme across all three fixes is that you need to understand the underlying data before you architect a solution on top of it. If you design a system assuming every question has one right answer, and the corpus actually contains contradictions, the system will eventually be forced into a confident single answer it can't fully support. If you design a system to state opinions as fact because the corpus is full of opinions, it will produce output that reads as authoritative when it shouldn't.

When that happens, the typical response is to say the model is 'hallucinating.' Often it isn't — it may be accurately reflecting content that exists in the retrieved documents. The actual defect is upstream: a design that assumed the wrong epistemic shape for the data, whether that's single-answer versus contested, or fact versus opinion. Auditing your corpus for these properties before building the retrieval and synthesis logic turns 'hallucination' from a mysterious model problem into a solvable data-modeling problem.

KEY POINTS

- Audit the corpus's epistemic type (single-answer, contested, or opinion-based) before choosing an answer format.
- Some reported hallucinations are the model faithfully reflecting retrieved content that the system's design assumed away.
- Matching the answer format to the true nature of the data is a data-modeling decision, made before generation, not a prompting trick applied after the fact.
- This audit step closes the loop with the earlier fixes: it's what tells you whether you need document curation, a clarification loop, calibrated confidence, or some combination of all three.

A pre-build corpus checklist

Before designing the retrieval and answer-synthesis logic, walk the corpus and answer these questions so the system's design matches the data's actual shape.

1. Does any topic have documents from multiple time periods? -> Need effective-date metadata and filtering.
2. Do any documents explicitly supersede others? -> Need a superseded_by link and exclusion at query time.
3. Can the same question have more than one valid answer depending on unstated scope? -> Need a clarification loop.
4. Is any part of the corpus opinion or interpretation rather than settled fact? -> Need explicit labeling in the synthesis prompt.
5. If a human read everything relevant, could they honestly say 'it depends'? -> The system must be allowed to say that too.

Glossary

Retrieval-Augmented Generation (RAG)

An architecture where a language model's answer is grounded in text retrieved from an external document store, rather than relying only on what the model learned during training.

Vector database

A database optimized for storing embeddings and finding the ones most similar to a query embedding, enabling search by meaning rather than exact keywords.

Embedding

A numeric vector representation of a piece of text such that texts with similar meaning end up close together in the vector space.

Semantic search

Searching for content by meaning, using embedding similarity, instead of matching exact keywords or phrases.

Clarification loop

A check performed before answering a question that detects whether the question is nonsensical or underspecified, and if so, prompts the user for more detail instead of guessing.

Unforced error

A preventable mistake in what content is allowed into the system, such as leaving a superseded document in a vector database, as opposed to a fundamental limitation of retrieval or generation.

Grounding

Constraining a language model's output to be based on specific supplied source text (the retrieved context) rather than on its general training knowledge.

Hallucination

A model output that states something ungrounded or false with unwarranted confidence; distinct from a model faithfully repeating contradictory or contested content it was given.

Document supersession metadata

Data attached to a document, such as its effective date and a pointer to the document that replaces it, used to identify and filter out obsolete versions.

Check yourself

Answer first, then open each one.

Two documents in a corpus each name a different team as the '2020 champion' for the same event, and both are technically defensible. Why does this happen, and whose fault is it?

It happens because 'champion year' is ambiguous between the season year and the calendar year the deciding game was played; both documents are answering slightly different unstated versions of the question, so it isn't a factual error in either one — it's an ambiguity the question itself never resolved.

A vector database returns both a 2019 policy and its 2024 replacement for the same query, and the LLM's answer ends up mixing outdated and current rules. Is this a retrieval bug, a generation bug, or something else?

It's a document curation problem, an 'unforced error.' Retrieval did its job by surfacing everything relevant, and generation did its job by synthesizing from what it was given; the fix is to attach effective-date and supersession metadata to documents and filter out superseded ones before they reach the model.

What two kinds of questions should trigger a clarification loop rather than an immediate answer, and why does each one cause problems if it isn't caught?

Nonsensical questions, which have no meaningful match in the domain and would otherwise force an answer out of loosely related retrieved text, and underspecified questions missing required scope (like a domain or time period), which would otherwise return an arbitrary answer based on whatever happens to be nearest in the vector space.

What does it mean for an AI's answer to be 'less than or equal to' the underlying data, and why does violating this look like a bigger problem than it is?

It means the system's certainty can't exceed what a human reading the same source documents would conclude; if the data honestly supports X, Y, or Z, the answer must reflect that multiplicity rather than picking one. Violating this produces an answer that sounds confident and authoritative while actually being arbitrary, which is more misleading than an answer that visibly hedges.

A team calls their RAG system's confident, single-answer response to a contested legal question a 'hallucination.' Why might that be the wrong diagnosis?

The model may be accurately reflecting content that exists in the retrieved documents; the real defect is that the system was designed to produce one settled answer for a corpus that is actually contested or opinion-based, so the fix is in the corpus audit and synthesis design, not necessarily in the model itself.

Where to go next

- Design a document metadata schema for a real corpus you work with, including `effective_date`, `superseded_by`, and source-authority fields, then add a retrieval-time filter that excludes superseded records.
- Prototype a lightweight clarification classifier — a single LLM call — that flags underspecified queries before they reach your vector search.
- Rewrite an existing RAG system's answer-synthesis prompt to enumerate multiple supported answers with citations instead of always returning exactly one.

- Audit a document collection you already index to identify sections that are contested or opinion-based, and check whether your current system labels them as such rather than presenting them as fact.
-

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.